AULA 11

Armazenamento local com AsyncStorage

Cross-Platform Application Development · React Native

useState: dados apenas em memória

FIAP

ESTADO (useState)

Os dados ficam na memória do aplicativo. Ao fechar o aplicativo, eles são perdidos.

ARMAZENAMENTO LOCAL

Os dados são gravados no dispositivo. Eles permanecem após fechar e reabrir o aplicativo.

O que é o AsyncStorage

Armazenamento persistente, simples e local ao aplicativo

1. Persistente: os dados permanecem após fechar o aplicativo
2. Chave-valor: cada dado é guardado sob um nome (a chave)
3. Assíncrono: leitura e gravação levam tempo e usam await
4. Local: fica no dispositivo e é exclusivo de cada aplicativo
5. Não criptografado: não é adequado para senhas ou dados sensíveis

Quando usar o AsyncStorage

FIAP

ADEQUADO PARA

- Preferências do usuário
- Listas simples, como a lista de compras
- Favoritos e histórico

NÃO RECOMENDADO PARA

- Senhas e dados sensíveis
- Grandes volumes de dados
- Dados que exigem consultas complexas

Os dados são removidos se o aplicativo for desinstalado.

O modelo chave-valor

Cada dado é gravado sob uma chave e recuperado por ela



Chave

Nome usado para gravar e para recuperar o dado

Valor

Conteúdo guardado, sempre em formato de texto

Semelhante a uma propriedade de objeto: a chave identifica o valor guardado.

Os três métodos principais

Todos pertencem ao objeto AsyncStorage

```
AsyncStorage.setItem(chave, valor)
```

Grava o valor sob a chave informada

```
AsyncStorage.getItem(chave)
```

Lê o valor da chave; devolve null se ela não existir

```
AsyncStorage.removeItem(chave)
```

Apaga a chave e o valor associado

Operações assíncronas

Gravar e ler exigem tempo, como o fetch

1. Cada método devolve uma Promise
2. Use await para aguardar o resultado
3. await só pode ser usado dentro de uma função async
4. getItem devolve null quando a chave ainda não existe

```
const salvo = await AsyncStorage.getItem('itens');
// salvo é um texto, ou null se nada foi gravado
```

Projeto base da aula

Obtenção do código inicial e instalação da biblioteca

```
git clone https://github.com/profrodrigomoraes/fiap-cpad-aula11.git
cd fiap-cpad-aula11
npm install
npx expo install @react-native-async-storage/async-storage
npx expo start
```

1. O projeto contém a lista de compras da Aula 7 (App.js)
2. Teste no emulador antes de começar as alterações

Código base: a lógica da lista

Arquivo App.js

```
const [texto, setTexto] = useState('');
const [itens, setItens] = useState([]);

function adicionarItem() {
  if (texto === '') {
    return;
  }
  setItens([...itens, texto]);
  setTexto('');
}
```

A lista existe apenas em memória: ao fechar o aplicativo, ela é perdida.

O AsyncStorage guarda apenas texto

Listas e objetos precisam ser convertidos

Ao salvar: JSON.stringify

```
const lista = ['arroz', 'feijão'];
const texto = JSON.stringify(lista);
// '["arroz","feijão"]'
```

Ao ler: JSON.parse

```
const texto = '["arroz","feijão"]';
const lista = JSON.parse(texto);
// ['arroz', 'feijão']
```

1. stringify: converte um array ou objeto em texto
2. parse: converte o texto de volta em array ou objeto

! Sem a conversão, o array seria gravado de forma incorreta

Passo 1: importar e salvar

A função adicionarItem passa a gravar a lista

```
import AsyncStorage from '@react-native-async-storage/async-storage';

async function adicionarItem() {
  if (texto === '') {
    return;
  }
  const novaLista = [...itens, texto];
  setItens(novaLista);
  setTexto('');
  await AsyncStorage.setItem('itens', JSON.stringify(novaLista));
}
```

1. A função passa a ser async para poder usar await
2. A nova lista é gravada como texto, com JSON.stringify

Passo 1: verificação

Testando o que foi implementado

1. Adicionar alguns itens à lista
2. Fechar o aplicativo e abri-lo novamente (ou recarregar com r)
3. A lista aparece vazia

Os dados foram gravados, mas o aplicativo ainda não os lê ao iniciar. Esse é o passo 2.

Passo 2: carregar ao abrir

Um useEffect lê a lista salva uma única vez

```
import { useState, useEffect } from 'react';

useEffect(() => {
  async function carregarItens() {
    const salvo = await AsyncStorage.getItem('itens');
    if (salvo !== null) {
      setItens(JSON.parse(salvo));
    }
  }

  carregarItens();
}, []);
```

1. getItem devolve null se nada foi salvo: por isso a verificação
2. JSON.parse converte o texto de volta em array

O fluxo completo

Ler ao abrir e gravar a cada alteração

Ao abrir o aplicativo



Ao adicionar um item



A leitura ocorre uma vez ao iniciar; a gravação ocorre a cada alteração da lista.

Passo 3: limpar a lista

Removendo os dados do estado e do armazenamento

```
async function limparLista() {
  setItens([]);
  await AsyncStorage.removeItem('itens');
}

<Button title="Limpar lista" onPress={limparLista} />
```

1. setItens([]) esvazia a lista exibida na tela
2. removeItem apaga a chave do armazenamento do dispositivo